6.3 · scikit-learn, métricas y validación

Una guía práctica y completa para dominar la API de scikit-learn, construir pipelines robustos, evaluar modelos con las métricas correctas y optimizar hiperparámetros con rigor científico.

MACHINE LEARNING SCIKIT-LEARN PYTHON



El patrón Estimator: tres métodos clave

Todos los objetos de scikit-learn comparten la misma interfaz. Independientemente del algoritmo, el flujo siempre es el mismo: instanciar, ajustar y predecir o transformar.



Instanciar

`modelo = RandomForestClassifier(n_estimators=100)`

Ajustar

`modelo.fit(X_train, y_train)`

Predecir

`modelo.predict(X_test)`

Los **transformadores** añaden el método `transform()`, mientras que los **predictores** implementan `predict()`. Un Pipeline combina ambos de forma secuencial.

¿Por qué usar Pipeline?

❌ Sin Pipeline (peligroso)

El preprocesamiento se aplica manualmente en cada paso. Es fácil cometer errores como escalar con estadísticas del test set, provocando **data leakage** y métricas optimistas.

- Código repetitivo y frágil
- Riesgo de contaminación train/test
- Difícil de mantener en producción

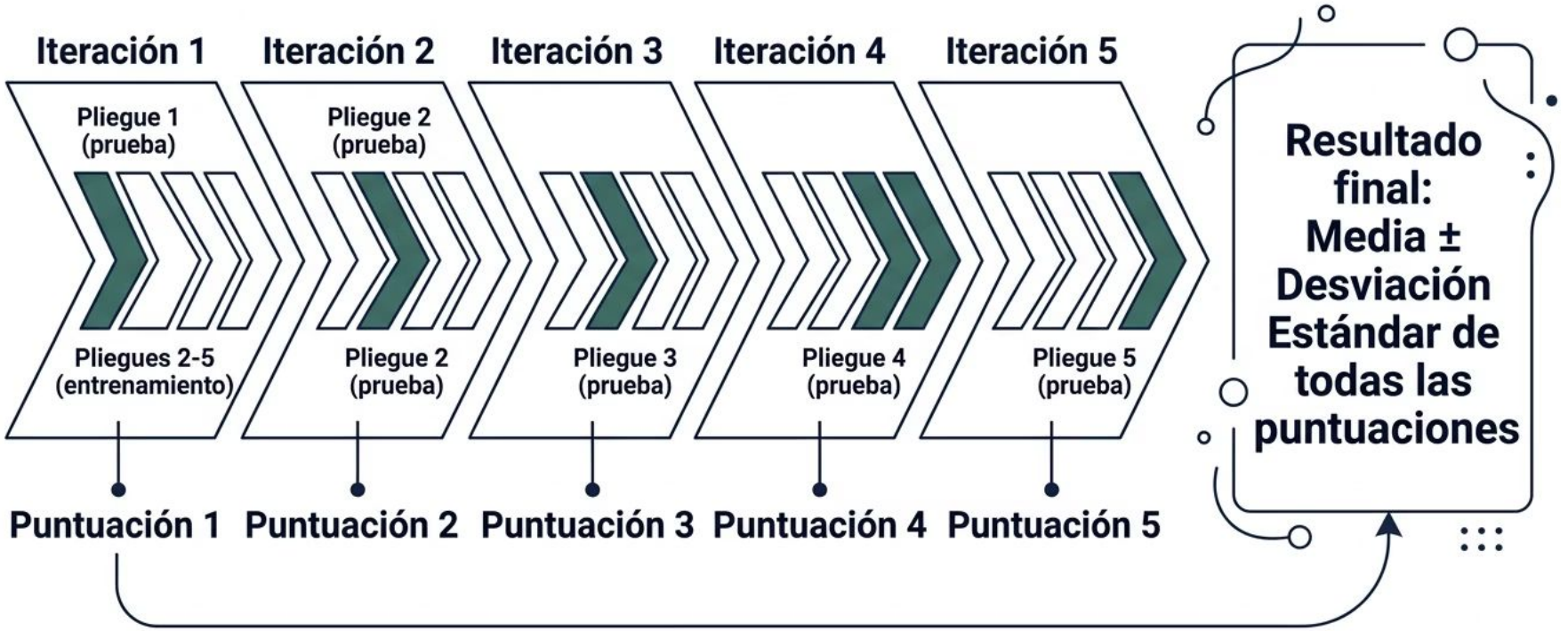
✅ Con Pipeline (correcto)

Todo el flujo —imputación, escalado, codificación y modelo— se encapsula en un objeto. El Pipeline aprende únicamente de los datos de entrenamiento y aplica las mismas transformaciones al test set.

- Código limpio y reproducible
- Compatible con cross-validation
- Listo para desplegar en producción

K-Fold Cross-Validation paso a paso

El conjunto de entrenamiento se divide en **k partes iguales (folds)**. El modelo se entrena k veces: cada iteración usa k-1 folds para entrenar y 1 fold para validar. El rendimiento final es la media de las k puntuaciones.



Un valor habitual es **k=5 o k=10**. Con k más alto se reduce el sesgo pero aumenta el coste computacional. La desviación estándar de las puntuaciones indica la estabilidad del modelo.

Comprendiendo MAE, RMSE y R^2

MAE — Error Absoluto Medio

Promedio de los errores absolutos. **Robusto ante outliers** porque no eleva al cuadrado. Interpretable: "en promedio, el modelo se equivoca en X unidades". Ideal cuando los errores grandes no son desproporcionadamente más graves.

RMSE — Raíz del Error Cuadrático Medio

Eleva al cuadrado los errores antes de promediar, **penalizando más los errores grandes**. Muy usado en competiciones de ML. Sensible a outliers. Sus unidades son las mismas que la variable objetivo.

R^2 — Coeficiente de determinación

Mide la **proporción de varianza explicada** por el modelo (0 a 1). Un R^2=0.85 significa que el modelo explica el 85% de la variabilidad de y. Puede ser negativo si el modelo es peor que predecir la media.

```
from sklearn.metrics import mean_absolute_error, root_mean_squared_error, r2_score

mae = mean_absolute_error(y_test, y_pred)
rmse = root_mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"MAE={mae:.2f} RMSE={rmse:.2f} R²={r2:.3f}")
```

Precisión, Recall y F1-Score

Precisión (Precision)

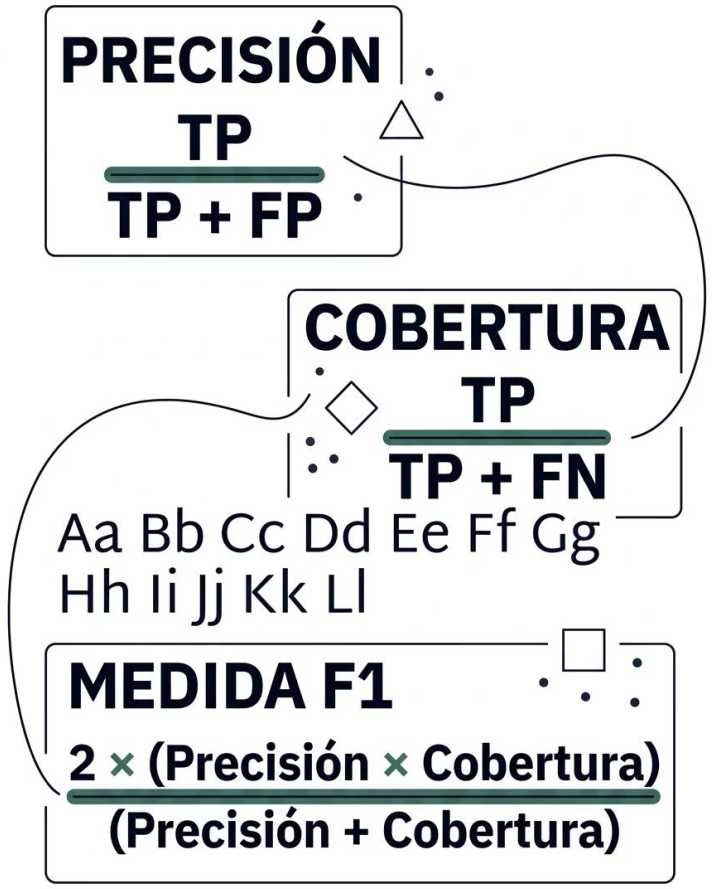
De todas las predicciones positivas, ¿cuántas son realmente positivas? Clave cuando los **falsos positivos son costosos** (ej. spam, diagnósticos invasivos).

Recall (Sensibilidad)

De todos los positivos reales, ¿cuántos detectamos? Crítico cuando los **falsos negativos son peligrosos** (ej. detección de enfermedades graves, fraude financiero).

F1-Score

Media armónica de precisión y recall. Equilibra ambas métricas. Ideal cuando se necesita un **compromiso único** entre las dos.



Matriz de confusión: anatomía de los errores

¿Qué nos dice la matriz?

Desglosa las predicciones en cuatro categorías para clasificación binaria:

TP (Verdadero Positivo)

Predijo positivo, era positivo ✅

TN (Verdadero Negativo)

Predijo negativo, era negativo ✅

FP (Falso Positivo)

Predijo positivo, era negativo ❌ (Error Tipo I)

FN (Falso Negativo)

Predijo negativo, era positivo ❌ (Error Tipo II)

Código en scikit-learn

```
from sklearn.metrics import (
    confusion_matrix,
    ConfusionMatrixDisplay
)

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=['Negativo','Positivo']
)
disp.plot(cmap='Blues')
```

Visualizar la matriz como mapa de calor permite detectar patrones de error sistemáticos, como confundir sistemáticamente una clase específica.

ROC–AUC: rendimiento independiente del umbral

La curva ROC representa la tasa de verdaderos positivos (TPR) frente a la tasa de falsos positivos (FPR) para todos los umbrales posibles. El **AUC (Área Bajo la Curva)** resume el rendimiento en un único número entre 0.5 (azar) y 1.0 (perfecto).

```
from sklearn.metrics import roc_auc_score, roc_curve
import matplotlib.pyplot as plt

# Probabilidades (no etiquetas)
y_prob = pipe.predict_proba(X_test)[:, 1]

auc = roc_auc_score(y_test, y_prob)
fpr, tpr, _ = roc_curve(y_test, y_prob)

plt.plot(fpr, tpr, label=f'AUC = {auc:.3f}')
plt.plot([0,1],[0,1],'--', color='gray')
plt.xlabel('FPR'); plt.ylabel('TPR')
plt.legend(); plt.show()
```

Usa `predict_proba()[:,1]` para obtener probabilidades, no etiquetas. Muchos clasificadores de scikit-learn lo implementan por defecto.

AUC ≈ 1.0

Clasificador casi perfecto

AUC ≈ 0.8

Rendimiento bueno

AUC ≈ 0.5

Equivalente a azar aleatorio

Diagnóstico: underfitting vs overfitting



Alto sesgo (Underfitting)

El modelo es demasiado simple para capturar los patrones. Tanto el error de entrenamiento como el de validación son **elevados**. Solución: modelo más complejo, más características, menos regularización.



Alta varianza (Overfitting)

El modelo memoriza el conjunto de entrenamiento. El error de entrenamiento es bajo pero el de validación es **muy superior**. Solución: más datos, regularización, dropout, o modelo más simple.



Equilibrio óptimo

Mínima diferencia entre error de entrenamiento y validación, ambos en un nivel aceptable. Se alcanza ajustando la capacidad del modelo y la cantidad de datos disponibles.

Guía de selección de métricas

Caso de uso	Métrica recomendada	Evitar	Razón
Predicción de precios	RMSE o MAE	R^2 solo	Interpretable en € o $
Clasificación equilibrada	Accuracy, F1 macro	—	Clases iguales en soporte
Clases desequilibradas	F1 weighted, ROC-AUC	Accuracy	Accuracy es engañosa
Coste asimétrico FN > FP	Recall	Precision	FN muy costoso
Coste asimétrico FP > FN	Precisión	Recall	FP muy costoso
Ranking / probabilidades	ROC-AUC	F1 binario	Umbral-independiente

S06 · EN EL REPOSITORIO DEL CURSO

Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

CUADERNO

S06_04_metricas_validacion — 45 min

Baseline, métricas, validación cruzada y fuga de datos

sprints/S06/notebooks/S06_04_metricas_validacion.ipynb

RETO

Reto del sprint — 20 min

Informe, README y ensayo (paso 5)

sprints/S06/guia/Reto_Sprint_06.md